

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: print(pd.isnull(np.nan))
print(pd.isnull(None))
print(pd.isna(np.nan))
print(pd.isna(None))                                # "na" - Not Available

print(pd.notnull(None))
print(pd.notnull(np.nan))
print(pd.notna(np.nan))
print(pd.notnull(3))

print()
print()
```

```
True
True
True
True
False
False
False
True
```

NA stands for "Not Available"

and represents missing, null, or undefined data in data analysis. In pandas, NA encompasses standard Python missing values like None, floating-point missing values like NaN (Not a Number), and pandas-specific missing data types.

```
In [3]: pd.isnull(pd.Series([1, np.nan, 7]))
```

```
Out[3]: 0    False
1     True
2    False
dtype: bool
```

```
In [4]: pd.notnull(pd.Series([1, np.nan, 7]))
```

```
Out[4]: 0     True
1    False
2     True
dtype: bool
```

Checking null on dataframe

```
In [5]: pd.isnull(pd.DataFrame({
    'Column A': [1, np.nan, 7],
    'Column B': [np.nan, 2, 3],
```

```
'Column C': [np.nan, 2, np.nan]
}))
```

```
Out[5]:
```

	Column A	Column B	Column C
0	False	True	True
1	True	False	False
2	False	False	True

Pandas Operations with Missing Values

Pandas manages missing values more gracefully than numpy. `nan`s will no longer behave as "viruses", and operations will just ignore them completely:

```
In [7]: pd.Series([1, 2, np.nan]).count()
```

```
Out[7]: np.int64(2)
```

```
In [8]: pd.Series([1, 2, np.nan]).sum()
```

```
Out[8]: np.float64(3.0)
```

```
In [9]: pd.Series([2, 2, np.nan]).mean()
```

```
Out[9]: np.float64(2.0)
```

Filtering missing data

```
In [10]: s = pd.Series([1, 2, 3, np.nan, np.nan, 4])
```

```
In [11]: pd.notnull(s)
```

```
Out[11]: 0    True
         1    True
         2    True
         3   False
         4   False
         5    True
         dtype: bool
```

```
In [14]: pd.notnull(s).sum() # getting no. of count for not null
```

```
Out[14]: np.int64(4)
```

```
In [15]: pd.isnull(s).sum() # getting no. of count for null
```

```
Out[15]: np.int64(2)
```

```
In [17]: s[pd.notnull(s)]           # retrieving not null values & also getting index n
```

```
Out[17]: 0    1.0
          1    2.0
          2    3.0
          5    4.0
          dtype: float64
```

Dropping null values - Better for repetitive task

```
In [18]: s
```

```
Out[18]: 0    1.0
          1    2.0
          2    3.0
          3    NaN
          4    NaN
          5    4.0
          dtype: float64
```

```
In [19]: s.dropna()
```

```
Out[19]: 0    1.0
          1    2.0
          2    3.0
          5    4.0
          dtype: float64
```

Dropping null values on DataFrames

You saw how simple it is to drop `na`s with a Series. But with `DataFrame`s, there will be a few more things to consider, because you can't drop single values. You can only drop entire columns or rows. Let's start with a sample `DataFrame` :

```
In [20]: df = pd.DataFrame({
          'Column A': [1, np.nan, 30, np.nan],
          'Column B': [2, 8, 31, np.nan],
          'Column C': [np.nan, 9, 32, 100],
          'Column D': [5, 8, 34, 110],
        })
```

```
In [21]: df
```

```
Out[21]:
```

	Column A	Column B	Column C	Column D
0	1.0	2.0	NaN	5
1	NaN	8.0	9.0	8
2	30.0	31.0	32.0	34
3	NaN	NaN	100.0	110

```
In [22]: df.shape
```

```
Out[22]: (4, 4)
```

```
In [23]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
#   Column   Non-Null Count  Dtype
---  -
0   Column A  2 non-null      float64
1   Column B  3 non-null      float64
2   Column C  3 non-null      float64
3   Column D  4 non-null      int64
dtypes: float64(3), int64(1)
memory usage: 260.0 bytes
```

```
In [24]: df.isnull()
```

```
Out[24]:
```

	Column A	Column B	Column C	Column D
0	False	False	True	False
1	True	False	False	False
2	False	False	False	False
3	True	True	False	False

```
In [26]: df.isnull().sum()
```

```
Out[26]: Column A    2
Column B    1
Column C    1
Column D    0
dtype: int64
```

The dropna will drop all the rows in which any null value is present:

```
In [27]: df.dropna()
```

```
Out[27]:
```

	Column A	Column B	Column C	Column D
2	30.0	31.0	32.0	34

```
In [32]: df.dropna(axis=0) # axis=0 means axis='rows'
```

```
Out[32]:
```

	Column A	Column B	Column C	Column D
2	30.0	31.0	32.0	34

```
In [33]: df.dropna(axis=1) # axis=1 means axis='columns'
```

Out[33]:

	Column D
0	5
1	8
2	34
3	110

In []:

In [34]: `df.dropna(how='any')` *# default behavior*

Out[34]:

	Column A	Column B	Column C	Column D
2	30.0	31.0	32.0	34

In [36]: `df.dropna(how='all')` *# dropna(how='all') drops a row only if every single*

Out[36]:

	Column A	Column B	Column C	Column D
0	1.0	2.0	NaN	5
1	NaN	8.0	9.0	8
2	30.0	31.0	32.0	34
3	NaN	NaN	100.0	110

df.dropna(how='all') drops a row only if every single value in that row is NaN (null).

This is different from the default behavior (how='any'), which drops a row if even one column contains a NaN.

In [38]: `df.dropna(thresh=3)`

Out[38]:

	Column A	Column B	Column C	Column D
0	1.0	2.0	NaN	5
1	NaN	8.0	9.0	8
2	30.0	31.0	32.0	34

thresh = 3, means row/column should have atleast 3 valid (not null)

In [41]: `df.dropna(thresh=3, axis='columns')` *#*

```
Out[41]:
```

	Column B	Column C	Column D
0	2.0	NaN	5
1	8.0	9.0	8
2	31.0	32.0	34
3	NaN	100.0	110

thresh = 3 , axis='columns' - means column should have atleast 3 valid (not null)

Filling null values

Sometimes instead than dropping the null values, we might need to replace them with some other value. This highly depends on your context and the dataset you're currently working. Sometimes a `nan` can be replaced with a `0` , sometimes it can be replaced with the `mean` of the sample, and some other times you can take the closest value. Again, it depends on the context. We'll show you the different methods and mechanisms and you can then apply them to your own problem.

```
In [43]:
```

```
s
```

```
Out[43]:
```

0	1.0
1	2.0
2	3.0
3	NaN
4	NaN
5	4.0

dtype: float64

Filling nulls with a arbitrary value

```
In [45]: s.fillna(0)    # fills null with "0"
```

```
Out[45]:
```

0	1.0
1	2.0
2	3.0
3	0.0
4	0.0
5	4.0

dtype: float64

```
In [46]: s.fillna(s.mean())
```

```
Out[46]:
```

0	1.0
1	2.0
2	3.0
3	2.5
4	2.5
5	4.0

dtype: float64

```
In [47]: s
```

```
Out[47]: 0    1.0
         1    2.0
         2    3.0
         3    NaN
         4    NaN
         5    4.0
         dtype: float64
```

Filling nulls with contiguous (close) values

The `method` argument is used to fill null values with other values close to that null one:

```
In [53]: s.ffmpeg()
```

```
Out[53]: 0    1.0
         1    2.0
         2    3.0
         3    3.0
         4    3.0
         5    4.0
         dtype: float64
```

```
In [54]: s.bfill()
```

```
Out[54]: 0    1.0
         1    2.0
         2    3.0
         3    4.0
         4    4.0
         5    4.0
         dtype: float64
```

bfill() & ffmpeg() can still leave null values at the extremes of the Series/DataFrame: below are examples

```
In [57]: pd.Series([np.nan, 3, np.nan, 9]).ffmpeg()
```

```
Out[57]: 0    NaN
         1    3.0
         2    3.0
         3    9.0
         dtype: float64
```

```
In [59]: pd.Series([1, np.nan, 3, np.nan, np.nan]).bfill()
```

```
Out[59]: 0    1.0
         1    3.0
         2    3.0
         3    NaN
         4    NaN
         dtype: float64
```

In [60]: df

Out[60]:

	Column A	Column B	Column C	Column D
0	1.0	2.0	NaN	5
1	NaN	8.0	9.0	8
2	30.0	31.0	32.0	34
3	NaN	NaN	100.0	110

In [66]: df.fillna({'Column A': 0, 'Column B': 99, 'Column C': df['Column C'].mean()}) #f

Out[66]:

	Column A	Column B	Column C	Column D
0	1.0	2.0	47.0	5
1	0.0	8.0	9.0	8
2	30.0	31.0	32.0	34
3	0.0	99.0	100.0	110

In [72]: df.ffill(axis='rows')

Out[72]:

	Column A	Column B	Column C	Column D
0	1.0	2.0	NaN	5
1	1.0	8.0	9.0	8
2	30.0	31.0	32.0	34
3	30.0	31.0	100.0	110

In [69]: df

Out[69]:

	Column A	Column B	Column C	Column D
0	1.0	2.0	NaN	5
1	NaN	8.0	9.0	8
2	30.0	31.0	32.0	34
3	NaN	NaN	100.0	110

In [71]: df.ffill(axis='columns')

```
Out[71]:
```

	Column A	Column B	Column C	Column D
0	1.0	2.0	2.0	5.0
1	NaN	8.0	9.0	8.0
2	30.0	31.0	32.0	34.0
3	NaN	NaN	100.0	110.0

Checking if there are NAs

The question is: Does this `Series` or `DataFrame` contain any missing value? The answer should be yes or no: `True` or `False`. How can you verify it?

Example 1: Checking the length

If there are missing values, `s.dropna()` will have less elements than `s`

```
In [75]:
```

```
s
```

```
Out[75]:
```

```
0    1.0
1    2.0
2    3.0
3    NaN
4    NaN
5    4.0
dtype: float64
```

```
In [74]:
```

```
s.dropna().count()      # count non-null
```

```
Out[74]:
```

```
np.int64(4)
```

```
In [76]:
```

```
missing_values = len(s.dropna()) != len(s)      # is there missing values?
missing_values
```

```
Out[76]:
```

```
True
```

More Pythonic solution `any`

The methods `any` and `all` check if either there's `any` True value in a Series or `all` the values are `True`. They work in the same way as in Python:

```
In [78]:
```

```
pd.Series([True, False, False]).any()      ## any true ?
```

```
Out[78]:
```

```
np.True_
```

```
In [79]:
```

```
pd.Series([True, False, False]).all()      # all true ?
```

```
Out[79]:
```

```
np.False_
```

```
In [80]: pd.Series([True, True, True]).all()
```

```
Out[80]: np.True_
```

```
In [81]: s.isnull()
```

```
Out[81]: 0    False
         1    False
         2    False
         3     True
         4     True
         5    False
         dtype: bool
```

The isnull() method returned a Boolean Series with True values wherever there was a nan:

So we can just use the any method with the boolean array returned: **.isnull().any()**

```
In [82]: pd.Series([1, np.nan]).isnull().any()
```

```
Out[82]: np.True_
```

```
In [87]: pd.Series([1, 2]).isnull().any()
```

```
Out[87]: np.False_
```

```
In [88]: s.isnull().any()
```

```
Out[88]: np.True_
```

```
In [89]: s.isnull().values
```

```
Out[89]: array([False, False, False,  True,  True, False])
```

```
In [90]: s.isnull().values.any()
```

```
Out[90]: np.True_
```